Known rows from the lesson’s guided examples: index 0, Sligo → Left 1, Right 3; index 2, Cork → Left null, Right null.
Check: Follow every non-null pointer to confirm it leads to the correct child. The complete eight-town tree is not reproduced in this lesson, so the remaining six rows must be checked against the original question diagram rather than guessed.

Q4. Step 1: Node 10 is at index 3. Its parent is node 20 at index 1.
Step 2: Since node 10 is a leaf, remove its parent's pointer to it.
CODE:
left[1] = -1;
// Search 10 again from root index 0:
// 40 → 20 → -1
Result: not found. The old value at data[3] is now unreachable from the root; a complete memory-management implementation would also mark the slot available for reuse.

Exit Ticket · Model answer

True. The left and right fields identify the positions of child nodes rather than storing duplicate child values.

Try the question again

Choose one question you got wrong. All correct? Choose the hardest.

1. Copy the whole original question:

2. Write a complete new answer in GREEN PEN. Show working if needed:

LESSON 022 · Trees and binary search trees with arrays

Name: _____ Class: _____ Date: _____

Time: _____ Teacher: _____

Work through the pages in order. Answer in BLACK PEN first. Check in GREEN PEN only after the Exit Ticket.

Do Now · Recall previous learning

Q1. What is an array, and how do you access an individual element?

Q2. What is the purpose of a pointer in a linked list?

Q3. Explain the difference between LIFO and FIFO. Name one data structure using each.

Q4. A graph has vertices A, B, C and D. Its undirected edges are A–B, A–C and C–D. Write its adjacency list.

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

I Do · Read new knowledge

Key vocabulary

Recall the term — tree

Recall the term — root

Recall the term — parent

Recall the term — child

Recall the term — leaf

Recall the term — binary tree

Recall the term — binary search tree

New learning

BST ordering — CODE:
smaller → left
larger → right — Each comparison chooses the next branch.

Search — CODE:
current = root;

while (current != -1 && data[current] != target)
{
if (target < data[current])
current = left[current];
else
current = right[current];
} — Search follows only one branch at each comparison.

Add — CODE:
if (value < data[parent])
left[parent] = newIndex;
else
right[parent] = newIndex; — The new index is attached to the correct null pointer.

Remove simple cases — CODE:
leaf → parent pointer = -1

one child → parent pointer = child index — Two-child removal is more involved; first secure leaf and one-child cases.

Worked example

RHS COMPUTING / OCR H446 / 1.4.2 LESSON 022 · DRAFT

BLACK PEN Answer all questions first. GREEN PEN Then: tick correct answers; rewrite mistakes.

Review & Improve · GREEN PEN

Now change to GREEN PEN. Find each original question. Tick correct answers; read the model and rewrite mistakes.

Page 1 onwards · Do Now

Q1. An array stores indexed elements. Access an element by its index, for example values[2].

Q2. It stores a reference or index identifying another node, allowing nodes to be linked together.

Q3. LIFO: last in, first out (stack). FIFO: first in, first out (queue).

Q4. A: B, C; B: A; C: A, D; D: C. Each undirected edge is represented at both ends.

You Do pages · Independent questions

Q1. Step 1: Assign an index to each node: 0=40, 1=20, 2=60, 3=10, 4=30.
CODE:
int[] data = {40,20,60,10,30};
int[] left = { 1, 3,-1,-1,-1};
int[] right = { 2, 4,-1,-1,-1};
Use -1 for a missing child. Root index = 0.

Q2. Target 10: 40 → left to 20 → left to 10. Found at index 3.
Target 60: 40 → right to 60. Found at index 2.
Target 99: 40 → right to 60 → right to -1. Not found.
CODE:
int current = 0;
int target = 10;
while (current != -1 && data[current] != target)
{
current = target < data[current]
? left[current] : right[current];
}
Console.WriteLine(current); // 3
Change target to 60 (output 2) and 99 (output -1).

Q3. Method: Record each town's array index first. In the Left and Right columns enter the index of its child, not the town name. Use null for an absent child when following the OCR table convention.

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

Search for 30

1. At index 0 — CODE:
30 < 40
current = left[0]
current = 1

2. At index 1 — CODE:
30 > 20
current = right[1]
current = 4

3. At index 4 — CODE:
data[4] = 30
found

4. Skipped nodes — CODE:
Did not inspect indexes 2 or 3.

We Do · Practise with guidance

Represent a BST in a 2D pointer table

OCR uses columns Left / Data / Right.

Complete one row

```
index 0: Sligo
left = ?
right = ?
```

```
index 0: Sligo
left = 1
right = 3
```

Leaf nodes

Cork is a leaf in the OCR tree.

Complete the same row

```
index 2: Cork
left = ?
right = ?
```

```
index 2: Cork
left = null
right = null
```

Search path

Search 50 in 40,20,60,50.

Complete the path

```
50 > 40 → ?
50 < 60 → ?
data = 50 → ?
```

```
50 > 40 → right
50 < 60 → left
data = 50 → found
```

Remove a leaf

Index 3 is a leaf and left[1] points to 3.

Complete the update

You Do · Work independently

BLACK PEN: Write your own answers. Do not look at the Review & Improve pages yet.

Question 1

Build a BST with parallel arrays
Represent 40 with children 20 and 60, then 20 with children 10 and 30.

Question 2

Search procedurally
Run the C# starter from I Do. Change the target to 10, 60, then 99. Predict the visit path and result. Then rewrite the search loop yourself.

Question 3

June 2024 H446/01 Q8 representation
Using the OCR towns BST, produce the Left/Data/Right table for all eight towns.

Question 4

Remove a leaf using a pointer change
Remove node 10 at index 3 from the example BST, then search for 10.

Exit Ticket · Answer before checking

In an array-based BST, left and right fields store indexes/pointers, not copies of the child data.